

Testbench Acceleration through Co-Emulation

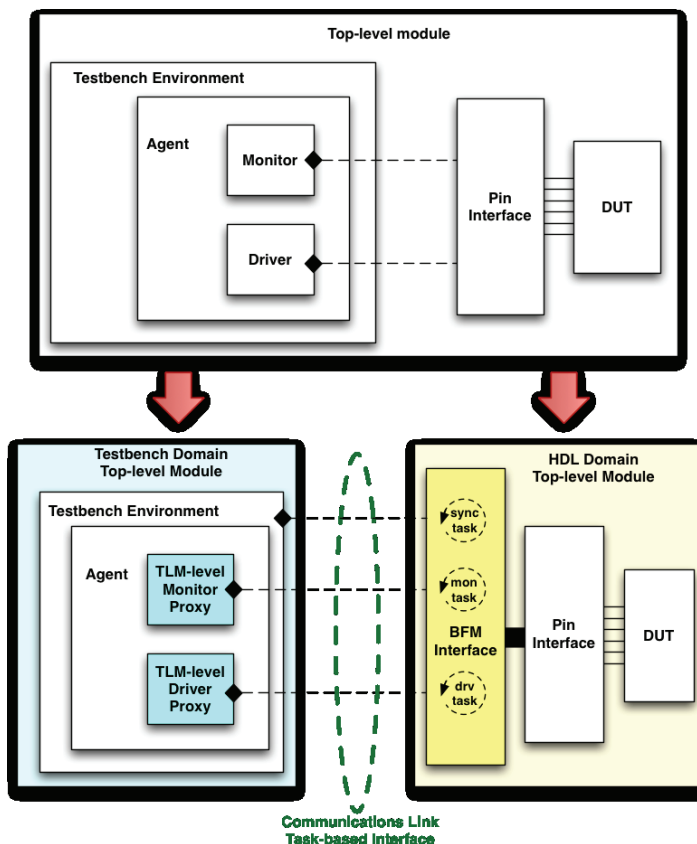
Emulation

Topic Overview

A hardware emulation platform, such as Mentor's Veloce, comprises a solution built upon specialized hardware to deliver high-performance model execution and provide orders of magnitude performance improvement over a traditional software simulator. For Veloce, the specialized hardware includes custom IC technology optimized for the needs of emulation and yet the simple use mode of software simulation.

Since models are compiled into hardware, they must be synthesizable. UVM testbenches rely heavily on object-oriented coding techniques, using SystemVerilog classes and other constructs that are not synthesizable and cannot be executed on an emulator - they must be executed on a software simulator. By combining software simulation for those parts of a testbench that are not synthesizable with hardware emulation for those parts that are synthesizable, you can gain much of the performance benefits of emulation while preserving the power and reuse benefits of UVM-based test environments.

This article describes a co-emulation (a.k.a. co-modeling) approach to writing testbenches that maximizes reuse by enabling truly single-source, fully IEEE 1800 SystemVerilog compliant testbenches that work interchangeably for both simulation and hardware-assisted acceleration. The content provided here is a basic introduction. Refer to the Mentor 3-series white paper on "Reducing Design Risk with Testbench Acceleration" ^[1] to gain a deeper understanding of the requirements and solutions presented.



Requirements

Executing a test using co-emulation, where a software simulator runs in tandem with an emulator, presents unique challenges. An emulator is a separate physical device that is executing behavior completely independently from the software simulator. It is connected to the simulator using a *transaction-based* communications link. This has the following implications:

- The synthesizable code that is to be run on the emulator must be completely separated from the non-synthesizable testbench code, placed in separate physical files as well as separate logical hierarchies. These two hierarchies are called the "testbench domain" and the "HDL domain".
- The testbench components that bridge the gap between the test environment and the physical environment - i.e. the so-called transactors like drivers and monitors - must be split into timed and untimed parts, and the parts placed in their respective domains. The untimed portion of each transactor resides in the testbench domain and becomes a proxy to the corresponding timed part in the HDL domain, referred to as BFM. The HDL BFM and its testbench proxy must interact at the transaction level over the emulator-simulator communications link.
- Any timing control statements including `#-delays`, `@-clock-edge` synchronizations, and fixed time-interval wait statements left in the testbench environment must be removed, as they impede co-emulation performance. As deemed necessary, these must be remodeled using clock synchronous behavior in the timed HDL domain.
- The link between the simulator and emulator is a physical communications link that deals in packets of data as opposed to transaction object handles. A mapping between packed data packets and transaction objects may therefore be required typically for object-oriented testbenches. Furthermore, in order to achieve highest possible performance the amount of traffic on the physical link must be minimized or optimized.

The above statements can be boiled down to the following three main requirements:

1. Verification code must be completely untimed, i.e. be free of explicit time advance statements. Abstract event synchronizations and waits for abstract events are allowed.
2. Untimed verification code must be separated from timed code into two domains.
3. No cross-domain signal or module references are allowed.

Further details can be found in the aforementioned Mentor 3-series white paper ^[1], providing a more elaborate explanation of UVM testbench acceleration architectural and performance fundamentals and considerations.

Methodology Overview

The goal here is to describe a methodology that not only meets the above requirements for co-emulation, but also does so using a single-source codebase, meaning that no code changes are needed to perform tests using either software-only simulation or co-emulation.

To arrive at this goal, different effort levels are required depending on one's starting point. If your testbench was constructed with the testbench architecture advocated and described throughout this UVM cookbook, relatively little effort is needed as the testbench already exhibits the dual TB-HDL domain structure with suitably partitioned transactors. Some additional work remains to ensure that all code in the HDL domain is synthesizable and uses performance optimal constructs such as proxy back-pointers. This entails in particular your driver and monitor transactor state machines, and clock and reset generation logic.

On the other hand, if the starting point is a traditional single top testbench, then significantly more work is required. In that case, the structural partitioning into two separated TB and HDL domains including the pertinent transactor splitting must be done "surgically". In particular, the transactors must be split into an untimed transaction-level proxy component in the TB domain (e.g. a SystemVerilog class like a `uvm_component` derivative) and a synthesizable BFM in the HDL domain (e.g. a SystemVerilog interface). User-defined tasks and functions - effectively representing transactions - are to be used to implement the transaction-level communication between the proxy and

the BFM ^[1], across the simulator-emulator boundary. An HDL BFM interface implements synthesizable tasks and functions to be called from the proxy via a virtual interface. Conversely, the BFM interface may initiate function calls back into its proxy via an object handle also referred to as back-pointer.

Lastly, irrespective of starting point, since code in the TB domain must be untimed, any of the aforementioned explicit timing control statements left in the testbench code must be semantically resolved, i.e re-modeled by synchronization with the timed HDL domain through the task-based proxy-BFM communication interface described above, or simply removed where possible.

Worked Example

The methodology pages contain a simple example. Another more comprehensive example starting from a single top level domain further illustrates the methodology and can be found [here](#).

References

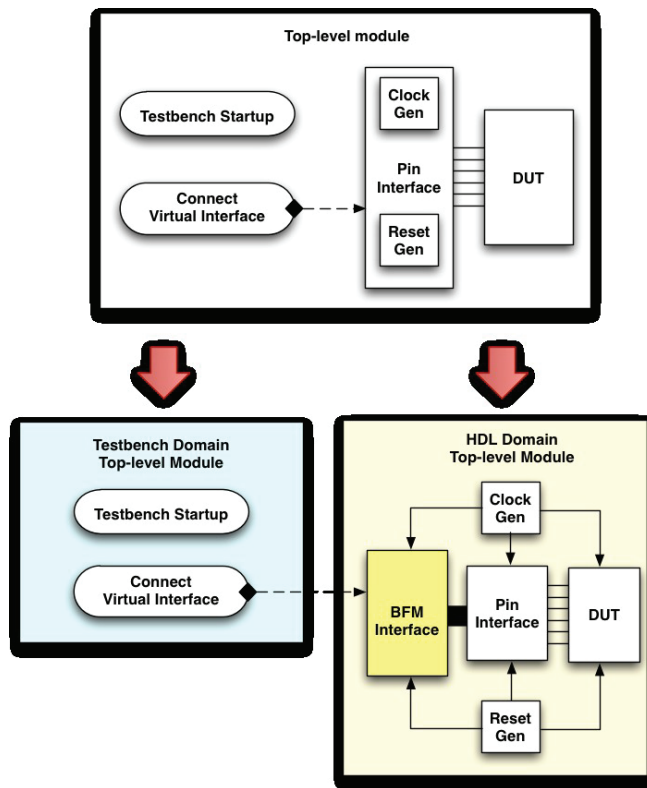
[1] <http://go.mentor.com/4xnvq>

Separate Top-Level Modules

Overview

Co-emulation is done by running two distinct synchronized model evaluations - one on a hardware emulator, and one on a software simulator. These two environments are physically separate and therefore require two domains, each with its own top-level module in separate files, called the HDL hierarchy and the testbench (TB) hierarchy. This dual-domain methodology generally follows the procedures detailed in the DualTop article and the architecture used throughout this cookbook defined in the testbench architecture article.

The actual use of an emulator imposes two additional related requirements. Firstly, while this cookbook overall does not require that the HDL domain be synthesizable, it becomes a requirement when targeting emulation. Secondly, all logic in the HDL domain must be synchronized to a clock-edge (a.k.a. be clock-synchronous). The remaining verification code residing in the testbench hierarchy executes in the software simulator that must be synchronized with the emulator, and must be untimed (e.g. "@(posedge clk)", "wait(1ns)", and "#10" are not allowed).



This article uses a **single top** bidirectional driver example as a simple testbench to illustrate the required partitioning between synthesizable HDL and untimed testbench domains. If your testbench already follows the architecture espoused throughout this cookbook, then you need to concern yourself only with ensuring that everything under your HDL top level be synthesizable:

```

module top_tb;

bus_if BUS();
gpio_if GPIO();
bidirect_bus_slave DUT(.bus(BUS), .gpio(GPIO));

// Free running clock
initial
begin
    BUS.clk = 0;
    forever begin
        #10 BUS.clk = ~BUS.clk;
    end
end

// Reset
initial
begin
    BUS.resetn = 0;
    repeat(3) begin
        @(posedge BUS.clk);
    end
    BUS.resetn = 1;
end

// UVM start up:
initial
begin
    uvm_config_db #(virtual bus_if)::set(null, "uvm_test_top", "BUS_vif" , BUS);
    run_test("bidirect_bus_test");
end

endmodule: top_tb

```

Interface and DUT Instantiation

Clock Gen

Reset Gen

Connect Virtual Interface

Testbench Startup

Testbench Domain

HDL Domain

Additionally, BFM interfaces should be created to contain the timed portions of the driver and monitor transactors.

```
interface bidirect_bus_driver_bfm (input bit      clk,
                                   input bit      resethn,
                                   output logic [31:0] addr,
                                   output logic [31:0] write_data,
                                   output logic      rnw,
                                   output logic      valid,
                                   input logic      ready,
                                   input logic [31:0] read_data,
                                   input logic      error);

    // pragma attribute bidirect_bus_driver_bfm partition_interface_xif

    // Synthesizable code implementing the timed transactor portion goes
    here ...

endinterface
```

The `// pragma` is used by the Veloce software to enable extended SystemVerilog code synthesis capabilities (XRTL) and must be applied to BFM.

Virtual Interface Connection

In the original testbench, the UVM transactors drive DUT signals through a pin-level interface. Veloce does not allow direct access to HDL signals from the testbench domain. A BFM interface must hence be created in the HDL domain (still a SystemVerilog interface), and the testbench domain communicates with the DUT indirectly through this BFM interface instead of directly through the pin-level interface.

Binding of a virtual interface to a concrete HDL-side interface can be done either in the testbench domain or the HDL domain. In the testbench domain, a hierarchical cross-reference to the HDL-side interface is used as shown below, and it is thus important that such binding be done in a top level area in the testbench domain with a global bird's eye view of the entire testbench topology configuration.

```
// Virtual interface handle now points to BFM interface instead of
pin-level interface
// NOTE: top_hdl.DRIVER is a hierarchical reference
uvm_config_db #(virtual bidirect_bus_driver_bfm)::set(null,
"uvm_test_top", "top_hdl.DRIVER" , top_hdl.DRIVER);
```

In order to align with normal DUT connection techniques, Veloce also conveniently supports virtual interface binding in the synthesizable HDL domain, right where the corresponding concrete BFM interface is instantiated. It requires the use of a virtual interface binding code block along with a simple pragma in adherence with the specific format given below.

```
module top_hdl;

    ...

    bidirect_bus_driver_bfm DRIVER( ... );

    // tbx vif_binding_block
    initial begin
```

```

import uvm_pkg::uvm_config_db;
uvm_config_db #(virtual bidirect_bus_driver_bfm)::set(null,
"uvm_test_top", $psprintf("%m.DRIVER") , DRIVER);
end

...

endmodule

```

Clock and Reset Generation

In addition to traditional synthesizable content, the HDL side should also contain clock generation and reset generation logic. Note that clock and reset code in conventional testbenches is often behavioral and normally not synthesizable. However, the Veloce emulator supports synthesis of a superset of RTL code called XRTL (eXtended RTL). According to XRTL coding guidelines, certain behavioral clock and reset generation blocks are permitted, requiring an extra pragma comment (*// tbx clkgen*). Even though the pragma comment states "clkgen", it applies for the reset block as well. Clock and reset signals must be generated through non-hierarchical access, so they should be declared at the top level and passed through ports into interfaces. As mentioned above, if the original clock and reset generation code is inside an interface, it must be moved directly to the top-level module.

```

// Free running clock
// tbx clkgen
initial begin
    clk = 0;
    forever begin
        #10 clk = ~clk;
    end
end

```

The statement "repeat (n) @ posedge clock" is not allowed in a Veloce/TBX clock generation specification block. This must be changed to the equivalent static delay, #50 in this case. Static parameters can be used here as well, for instance imported from a shared parameters package. Veloce XRTL also supports variable clock delays, the details of which are not complex but beyond the scope of this article.

```

// Reset
// tbx clkgen
initial begin
    resetn = 0;
    #50 resetn = 1;
end

```

Testbench Domain

Here is the complete testbench domain top-level code, with explanation to follow.

```
// Top level test bench module
module top_tb;

import uvm_pkg::*;
import bidirect_bus_pkg::*;

// UVM start up:
initial
begin
    run_test("bidirect_bus_test");
end

endmodule: top_tb
```

Testbench Startup

The code that starts the class-based UVM testbench environment remains in the testbench domain.

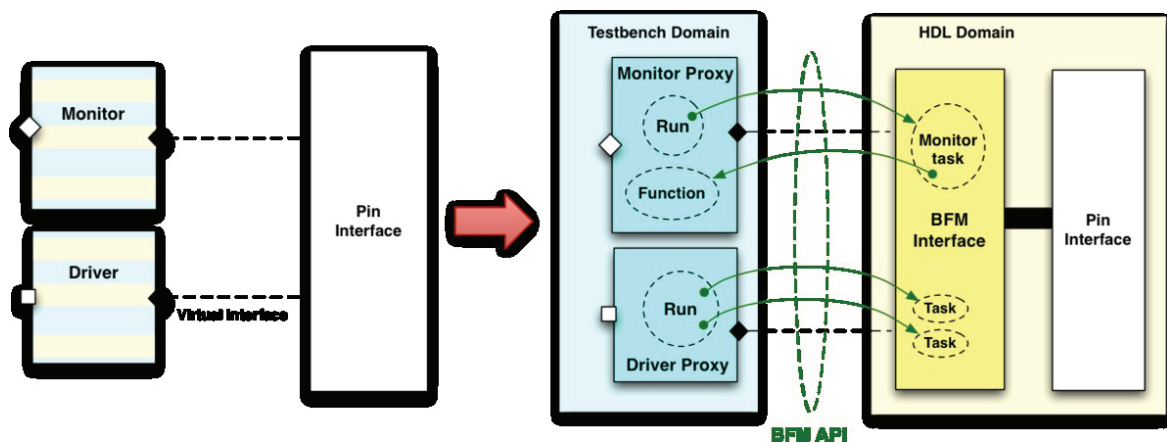
```
run_test("bidirect_bus_test");
```

Shared Parameters

Any parameters that are common between the testbench domain and the HDL domain should be declared in a shared parameter package imported by both top-level modules. If parameters are used only in the HDL domain, it is not necessary to use a shared parameter package.

Split Transactors

Overview



Driver and monitor transactors contain a mixture of transaction-level code to communicate with the testbench, and clock-driven HDL signal accessing code to communicate with the DUT through a virtual interface. As for the top-level module, the portions of transactor code that are timed and access signals must be identified, and separated from the untimed part that manipulates transaction objects.

In general, a transactor follows a common pattern:

1. Drivers - Get a transaction;
2. Drivers and Monitors - Interact with DUT signals through a virtual interface;

3. Drivers and Monitors - Send a transaction out through a sequence item port or an analysis port.

The second point, interacting with DUT signals, needs to be partitioned from the code that sends/receives transactions. This signal-level code is to be compiled and executed on the emulator. Data transfer between the emulator and the simulator takes place over a communications link. The communication semantics are modeled as tasks and functions declared in a BFM interface in the HDL domain (still using the SystemVerilog interface construct). The testbench-side transactor object now acts as a proxy for the BFM, calling these BFM tasks and functions (a.k.a. BFM API) to execute interface protocol cycles (i.e. indirectly drive and sample signals), set configuration parameters or get status information. Any data that needs to be exchanged between the BFM and its proxy is passed as task and function input or output arguments.

When following the dual domain testbench architecture used throughout this cookbook, the split between proxy and BFM is implied and has been done already. However, there is still work to do because the BFM and all interactions with the BFM must be with synthesizable code. This includes any arguments to functions and tasks that may be called to/from the proxy and BFM. Object handles can no longer be used to pass information. Instead, individual arguments can be used or a packed struct could be used. More information on this topic is available in the article on Defining the BFM API.

In addition to the testbench domain initiating activity inside a BFM interface by the BFM proxy calling BFM tasks and functions, it is also possible for the BFM to initiate communication with its proxy by calling non-blocking methods in the proxy through a back-pointer.

Original Driver Code

The following code, from the **single top** bidirectional driver example, shows typical UVM driver code written as a class:

```
class bidirect_bus_driver extends uvm_driver #(bus_seq_item);

virtual bus_if BUS;

task run_phase(uvm_phase phase);
    bus_seq_item req;
    bus_seq_item rsp;
    // Default conditions:
    BUS.valid <= 0;
    BUS.rnw <= 1;
    // Wait for reset to end
    @(posedge BUS.resetn);
    forever
    begin
        seq_item_port.get(req); // Start processing req item
        repeat(req.delay) begin
            @(posedge BUS.clk);
        end
        BUS.valid <= 1;
        BUS.addr <= req.addr;
        BUS.rnw <= req.read_not_write;
        if(req.read_not_write == 0) begin
            BUS.write_data <= req.write_data;
        end
        while(BUS.ready != 1) begin
            @(posedge BUS.clk);
        end
        // At end of the pin level bus transaction
        // Copy response data into the rsp fields:
        $cast(rsp, req.clone()); // Clone the req
        rsp.set_id_info(req); // Set the rsp id = req id
        if(rsp.read_not_write == 1) begin
            rsp.read_data = BUS.read_data; // If read - copy returned read data
        end
        rsp.error = BUS.error; // Copy bus error status
        BUS.valid <= 0; // End the pin level bus transaction
        seq_item_port.put(rsp); // put returns the response
    end
endtask: run_phase

endclass: bidirect_bus_driver
```

Testbench Domain

HDL Domain

This must thus be split into two partitioned parts inside separate files, with the timed interface protocol ultimately executed on the emulator and the TLM-based portion run in the software-based UVM simulation environment.

Testbench Domain Code

The testbench side of the transactor should deal with any TLM-level communication to/from the class-based testbench (e.g. `seq_item_port` and `analysis_port`), and any TLM-level manipulation of transaction objects.

```
class bidirect_bus_driver extends uvm_driver #(bus_seq_item);

virtual bidirect_bus_driver_bfm BFM; //Transactor now uses BFM interface
                                     //instead of pin interface

task run_phase(uvm_phase phase);
    bus_seq_item req;
    bus_seq_item rsp;

    // Call BFM task here to wait for reset to end
    BFM.wait_for_reset();

    forever
    begin
        bus_seq_item_s req_s, rsp_s;

        seq_item_port.get(req); // Start processing req item

        //Extract relevant transaction data into packed format:
        bus_seq_item_converter::from_class(req, req_s);

        // Call BFM task here to execute transaction
        // Pass required transaction data to BFM as arguments to the task
        //Get response from BFM through task output argument
        BFM.do_item(req_s, rsp_s);

        //Convert from packed format back to transaction object
        bus_seq_item_converter::to_class(rsp, rsp_s);

        rsp.set_id_info(req); // Set the rsp id = req id
        seq_item_port.put(rsp); // put returns the response
    end
endtask: run_phase

endclass: bidirect_bus_driver
```

Virtual Interface Change

The original transactor performed direct signal access using a handle to the pin interface 'bus_if'. The updated transactor now performs all timed and bus signal operations through the BFM interface:

```
virtual bidirect_bus_driver_bfm BFM;
```

Replace Timed Portions with Calls to BFM Tasks and Functions

The timed code that directly accesses signals is moved into tasks in the BFM interface. The updated transactor now calls these tasks through the BFM virtual interface:

```
// Wait for reset to end
BFM.wait_for_reset();

// Call BFM task here to perform transaction
// Pass required transaction data to BFM as arguments to the task
// Get response from BFM from task argument
BFM.do_item(req_s, rsp_s);
```

The mapping between data from the transaction object format ('req' and 'rsp' in the example) and a synthesizable packed format ('req_s' and 'rsp_s' in the example) is discussed in the article on Defining the BFM API.

HDL Domain Code

The timed, signal-level code of the driver is moved into the BFM interface. This code should be written in a synthesizable style and must adhere to the Veloce SystemVerilog XRTL subset. For instance, Veloce XRTL requires task and function arguments to be input or output only - no inout and ref arguments.

```
interface bidirect_bus_driver_bfm (input bit      clk,
                                  input bit      resetn,
                                  output logic [31:0] addr,
                                  output logic [31:0] write_data,
                                  output logic      rnw,
                                  output logic      valid,
                                  input  logic      ready,
                                  input  logic [31:0] read_data,
                                  input  logic      error);

// pragma attribute bidirect_bus_driver_bfm partition_interface_xif

import bidirect_bus_shared_pkg::bus_seq_item_s;

initial begin
    // Default conditions:
    valid <= 0;
    rnw <= 1;
end

task wait_for_reset(); // pragma tbx xtf
    @(posedge resetn);
endtask

task do_item(input bus_seq_item_s req, output bus_seq_item_s rsp); // pragma tbx xtf
    @(posedge clk);
    repeat(req.delay-1) begin
        @(posedge clk);
    end
    valid <= 1;
    addr <= req.addr;
    rnw <= req.read_not_write;
    if(req.read_not_write == 0) begin
        write_data <= req.write_data;
    end
    while(ready != 1) begin
        @(posedge clk);
    end
    // At end of the pin level bus transaction
    // Copy response data into the rsp fields:
    rsp = req; // Clone the req
    if(req.read_not_write == 1) begin
        rsp.read_data = read_data; // If read - copy returned read data
    end
    rsp.error = error; // Copy bus error status
    valid <= 0; // End the pin level bus transaction
endtask

endinterface
```

Pragma Comments

Veloce requires special pragma comments for BFM tasks and functions that are called from the proxy in the testbench domain. Firstly, after the interface declaration you must add the following pragma comment:

```
// pragma attribute bidirect_bus_driver_bfm partition_interface_xif
```

Secondly, on the declaration line of the tasks and functions called from the proxy you must add the following pragma comment:

```
// pragma tbx xtf
```

Note that Veloce XRTL does not permit these BFM API tasks and functions to be called locally within the HDL domain, so only annotate BFM API routines with this pragma. If you really need to reuse a task or function for such dual purpose, you can often do so by coding it without a pragma for local HDL domain usage, and also putting a call inside a wrapper task/function with a pragma.

Initial Values and Synchronization

The two default value assignments have been moved into an initial block in the BFM interface. Veloce XRTL extends normal RTL synthesis rules by allowing execution of initial blocks:

```
initial begin
    // Default conditions:
    BUS.valid <= 0;
    BUS.rnw <= 1;
end
```

The line that waits for reset to end is a timing control statement, so it must be moved to a task:

```
task wait_for_reset(); // pragma tbx xtf
    @(posedge BUS.resetn);
endtask
```

Main Transactor Code

The original transactor code started with 'repeat (req.delay) begin ...'. Veloce XRTL requires that all timed code be clock-synchronous, and in particular time-consuming BFM tasks called from the testbench domain must start with a clock edge synchronization. The original 'repeat' loop has been changed accordingly in the BFM task to the following equivalent code (which is valid as the delay parameter is constrained in the transaction class to be at least 1):

```
@(posedge BUS.clk);
repeat(req.delay-1) begin
    @(posedge BUS.clk);
end
```

In the next code section, the attributes of the transaction request (addr, read_not_write, write_data, etc.) are applied to the pin interface. These attributes come from the input argument 'req' of the 'do_item' BFM task, rather than from the initial UVM transaction object directly. Assignments should be converted to either non-blocking or blocking, following established best coding practices for synthesis.

```
BUS.valid <= 1;
BUS.addr <= req.addr;
BUS.rnw <= req.read_not_write;
if(req.read_not_write == 0) begin
    BUS.write_data <= req.write_data;
end
while(BUS.ready != 1) begin
    @(posedge BUS.clk);
end
```

Response Handling

In the final code section, the response is sent back to the transactor proxy in the testbench domain, via the 'do_item' BFM task output argument 'rsp':

```
// At end of the pin level bus transaction
// Copy response data into the rsp fields:
rsp = req; // Clone the req
if(req.read_not_write == 1) begin
    rsp.read_data = BUS.read_data; // If read - copy returned read data
end
rsp.error = BUS.error; // Copy bus error status
```

Back Pointers

Overview

In the original **single top** bidirectional driver example, all driver activity is initiated from the testbench domain. Sometimes it is more natural and efficient to initiate activity from the HDL domain, particularly in monitor transactors to send analysis transactions out to the testbench domain, or in pipelined drivers to send back responses. Veloce supports "back-pointers" in the HDL domain, which are simply handles (back) to class-based components in the testbench domain. Class methods can be called from the HDL domain through such back-pointers to initiate communication.

Below is a version of the driver that uses 'get_next_item()' and 'item_done()' to process transactions:

```
class bidirect_bus_driver extends uvm_driver #(bus_seq_item);

    bus_seq_item req;

    virtual bus_if BUS;

    task run_phase(uvm_phase phase);

        // Default conditions:
        BUS.valid <= 0;
        BUS.rnw <= 1;
        // Wait for reset to end
        @(posedge BUS.resetn);
        forever
        begin
            seq_item_port.get_next_item(req); // Start processing req item
            repeat(req.delay) begin
                @(posedge BUS.clk);
            end
            BUS.valid <= 1;
            BUS.addr <= req.addr;
            BUS.rnw <= req.read_not_write;
            if(req.read_not_write == 0) begin
                BUS.write_data <= req.write_data;
            end
            while(BUS.ready != 1) begin
                @(posedge BUS.clk);
            end
            // At end of the pin level bus transaction
            // Copy response data into the req fields:
            if(req.read_not_write == 1) begin
                req.read_data = BUS.read_data; // If read - copy returned read data
            end
            req.error = BUS.error; // Copy bus error status
            BUS.valid <= 0; // End the pin level bus transaction
            seq_item_port.item_done(); // End of req item
        end
    endtask: run_phase

endclass: bidirect_bus_driver
```

Testbench Domain

HDL Domain

Testbench Domain

Timed code must be extracted and moved from the testbench domain to a BFM interface in the HDL domain as before, yet in this case the BFM will be "in control". The BFM's proxy in the testbench domain merely starts up the BFM thread(s) in its 'run()' task, and the transaction-level communication back from the BFM to the proxy is handled by designated (and appropriately named) functions:

```
class bidirect_bus_driver extends uvm_driver #(bus_seq_item);

bus_seq_item req;

virtual bidirect_bus_driver_bfm BFM;

function void end_of_elaboration_phase(uvm_phase phase);
    BFM.proxy = this;
endfunction: end_of_elaboration_phase

task run_phase(uvm_phase phase);
    BFM.run();
endtask: run_phase

task try_next_item(output bus_seq_item_s req_s, output bit success);
    seq_item_port.try_next_item(req); // Start processing req item
    success = (req != null);
    if (success)
        bus_seq_item_converter::from_class(req, req_s);
endtask: try_next_item

function void item_done(input bus_seq_item_s req_s);
    bus_seq_item req;
    bus_seq_item_converter::to_class(req, req_s);
    this.req.copy(req);
    seq_item_port.item_done(); // End of req item
endfunction: item_done

endclass: bidirect_bus_driver
```

Setting the Back-Pointer

The back-pointer to the proxy in the testbench domain is declared inside the corresponding BFM interface in the HDL domain:

```
interface bidirect_bus_driver_bfm (...);
    // pragma attribute bidirect_bus_driver_bfm partition_interface_xif
    ...

    bidirect_bus_pkg::bidirect_bus_driver proxy;

    ...
endinterface
```

The BFM's back-pointer is then assigned somewhere after binding the BFM virtual interface^[1] but before the start of the UVM run phase. For this example it must be between the connect and run phases, i.e. in the end-of-elaboration phase:

```
function void end_of_elaboration_phase(uvm_phase phase);
    BFM.proxy = this;
endfunction: end_of_elaboration_phase
```

Start Transaction Processing

Here, the main flow of control is based in the BFM, so the proxy merely initiates and then yields to the BFM:

```
task run_phase(uvm_phase phase);  
    BFM.run();  
endtask: run_phase
```

Defining Back-Pointer Methods

The untimed behavior of the driver in the testbench domain that manipulates class-based transactions is moved into functions or zero-time tasks to be called by the driver BFM. The method 'try_next_item()' of the UVM 'seq_item_port', even though it is zero-time, is technically (and seemingly wrongful!) a task, so it must be wrapped in a task rather than a function. Since the BFM needs to know if the call actually provided an item, a success status bit is further added:

```
task try_next_item(output bus_seq_item_s req_s, output bit success);  
    seq_item_port.try_next_item(req); // Start processing req item  
    success = (req != null);  
    if (success)  
        bus_seq_item_converter::from_class(req, req_s);  
endtask: try_next_item  
  
function void item_done(input bus_seq_item_s req_s);  
    bus_seq_item req;  
    bus_seq_item_converter::to_class(req, req_s);  
    this.req.copy(req);  
    seq_item_port.item_done(); // End of req item  
endfunction: item_done
```

HDL Domain

```

interface bidirect_bus_driver_bfm (input bit      clk,
                                  input bit      resetn,
                                  output logic [31:0] addr,
                                  output logic [31:0] write_data,
                                  output logic      rnv,
                                  output logic      valid,
                                  input logic      ready,
                                  input logic [31:0] read_data,
                                  input logic      error);
// pragma attribute bidirect_bus_driver_bfm partition_interface_xif
import bidirect_bus_shared_pkg::bus_seq_item_s;
import bidirect_bus_pkg::bidirect_bus_driver;

bidirect_bus_driver proxy;

bus_seq_item_s req;

bit go;

function void run(); // pragma tbx xtf
// Default conditions:
valid <= 0;
rnv <= 1;
go = 1;
endfunction

initial begin
// Wait for reset to end and go signal from run()
wait(resetn & go);
@(posedge clk);
forever
begin
bit success;
proxy.try_next_item(req, success); // Start processing req item
while (!success) begin
@(posedge clk);
proxy.try_next_item(req, success); // Start processing req item
end
@(posedge clk);
repeat(req.delay-1) begin
@(posedge clk);
end
valid <= 1;
addr <= req.addr;
rnv <= req.read_not_write;
if(req.read_not_write == 0) begin
write_data <= req.write_data;
end
while(ready != 1) begin
@(posedge clk);
end
// At end of the pin level bus transaction
// Copy response data into the req fields:
if(req.read_not_write == 1) begin
req.read_data = read_data; // If read - copy returned read data
end
req.error = error; // Copy bus error status
valid <= 0; // End the pin level bus transaction
proxy.item_done(req); // End of req item
end
end
endinterface

```

Declaring the Back-Pointer

The declaration of a back-pointer from the BFM interface in the HDL domain to its corresponding proxy in the testbench domain is as shown earlier above ^[2].

Main Run Thread

Back-pointers are most helpful when the bulk of processing is done on the HDL side, making it more natural for this side to be an initiator. This pattern works particularly well for monitor transactors ^[3]. In this example of the bidirectional driver, the main forever loop is implemented on the HDL side, and it communicates with the testbench side to get a request transaction and send back the response:

```

task run_phase(uvm_phase phase); // pragma tbx xtf
// Other code not shown...

forever begin
bit success;

proxy.try_next_item(req, success); // Start processing req item

while (!success) begin
@(posedge BUS.clk);
proxy.try_next_item(req, success); // Start processing req item
end

// Process transaction, get response ...

```

```

    proxy.item_done(req); // End of req item
end
endtask

```

References

- [1] https://verificationacademy.com/cookbook/Emulation/SeparateTopLevels#Virtual_Interface_Connection
- [2] https://verificationacademy.com/cookbook/Emulation/BackPointers#Setting_the_Back-Pointer
- [3] <https://verificationacademy.com/patterns-library/implementation-patterns/analysis-patterns/bfm-notification-pattern>

Defining an API

Overview

As the timed portion of the traditional UVM transactor must be moved over to the HDL domain, the pertinent lines of code must be extracted and rearranged into synthesizable FSMs and associated logic inside a BFM interface, a SystemVerilog interface. This entails in particular the creation of tasks and functions in that BFM interface to effectively define a transaction-based API for the BFM, accessible from the testbench domain. When designing the BFM API, consider that the partitioned testbench and HDL domains are running on separate platforms, workstation and emulator, and therefore communication between them can have a big impact on run-time performance. Consequently, communication should be kept relatively infrequent and data-rich, as opposed to frequency and "data-light" (e.g. imagine a Ferrari driving through a large city's downtown intersections, hitting every red light).

The data involved in the HDL domain to drive and monitor the DUT's pin interfaces is typically associated with stimulus and analysis transactions in the testbench domain. These transactions are generally class-based and model an abstraction of the pin-level DUT data. They are optimized for class-based communication and manipulation. Yet, the HDL domain does not support class objects, and hence transaction data attributes from the testbench domain may need to be converted into synthesizable DUT-compatible data and vice-versa.

BFM Task and Function Arguments

When the cycle-based signal-level portion of transactor code is moved to a BFM interface, transaction data needs to be passed between the BFM and its proxy via synthesizable task arguments. Stimulus data enters the HDL BFM from the testbench domain and analysis data exits the BFM to the testbench domain. The BFM implementation ultimately determines which pieces of transaction data are manipulated in the HDL domain and must be communicated via the BFM task and function API.

Given below is the main BFM task for the **single top** bidirectional driver example, declared inside the driver BFM interface in the HDL domain and called from the driver proxy in the testbench domain:

```

task do_item(input bus_seq_item_s req, output bus_seq_item_s rsp); //
pragma tbx xtf
    @(posedge clk);
    repeat(req.delay-1) begin
        @(posedge clk);
    end

    valid <= 1;
    addr <= req.addr;
    rnw <= req.read_not_write;
    if(req.read_not_write == 0) begin

```

```

    write_data <= req.write_data;
end

while(ready != 1) begin
    @(posedge clk);
end

// At end of the pin level bus transaction
// Copy response data into the rsp fields:
rsp = req; // Clone the req
if(req.read_not_write == 1) begin
    rsp.read_data = read_data; // If read - copy returned read data
end
rsp.error = error; // Copy bus error status

valid <= 0; // End the pin level bus transaction
endtask

```

This task takes the transaction attributes 'addr', 'read_not_write', 'write_data', and 'delay' from a request transaction, and sets the attributes 'error' and 'read_data' of a response transaction.

If there are just a few data attributes, like address and data, and already in a packed format, these are readily passed as straight individual task arguments. On the other hand, if there are many pertinent transaction attributes, it may be better suited to define a synthesizable packed struct container and pass all attributes together via a single struct task argument. For the current driver example, the BFM requires four request and two response data attributes, so a struct is defined accordingly:

```

package bidirect_bus_pkg;

typedef struct packed {
    // Request fields
    logic [31:0] addr;
    logic [31:0] write_data;
    bit         read_not_write;
    int         delay;

    // Response fields
    bit         error;
    logic [31:0] read_data;
} bus_seq_item_s;

// Rest of package not shown...

endpackage: bidirect_bus_pkg

```

The BFM task 'do_item(...)' utilizes this packed struct type to exchange data with its proxy. Note that Veloce supports input and output arguments - inouts are syntactic sugar while refs are a bit like pointers and not generally synthesizable. The 'do_item' task arguments are hence as follows:

```

task do_item(input bus_seq_item_s req, output bus_seq_item_s rsp);

```

Converting Between Testbench Transactions and HDL BFM Data

The mapping between higher level transaction objects and low level pin wiggles - sometimes referred to as data packing/unpacking - is conventionally done either directly as part of a testbench component's `run_phase()` thread(s), or indirectly by calling helper functions. For the split proxy-BFM pairs it can still be done this way, i.e. in a BFM's proxy, or alternatively it can be delegated to an external converter class as follows:

```
// Delegate class that converts between bus_seq_item transaction and
// synthesizable packed struct

class bus_seq_item_converter;

    static function void from_class(input bus_seq_item t, output
bus_seq_item_s s);
        s.addr            = t.addr;
        s.write_data      = t.write_data;
        s.read_not_write  = t.read_not_write;
        s.delay           = t.delay;
        s.error           = t.error;
        s.read_data       = t.read_data;
    endfunction

    static function void to_class(output bus_seq_item t, input
bus_seq_item_s s);
        t = new();
        t.addr            = s.addr;
        t.write_data      = s.write_data;
        t.read_not_write  = s.read_not_write;
        t.delay           = s.delay;
        t.error           = s.error;
        t.read_data       = s.read_data;
    endfunction

endclass: bus_seq_item_converter
```

This converter class is then used to map the original bus sequence item object into a corresponding struct, and vice versa:

```
bus_seq_item_converter::from_class(req, req_s);
BFM.do_item(req_s, rsp_s);
bus_seq_item_converter::to_class(rsp, rsp_s);
```

Example

Overview

This article steps through the process of converting a comprehensive traditional **single top** UVM example testbench to an equivalent one with a dual domain partitioned structure that is ready for co-emulation with Veloce. This emulation-ready testbench adheres to the methodology advocated throughout this cookbook, enabling testbench reuse for both hardware-assisted acceleration and conventional software simulation. For your own testbench, if it exhibits the dual domain architecture from inception, some of the steps described here are not applicable since you have effectively performed them already. Specifically, this includes top level partitioning and transactor splitting. Some work may still be needed to ensure clocks and resets are ready for emulation, and everything in the HDL domain including BFM function and task call arguments are synthesizable.

Split Top Level

The first thing to be done is to partition the single top level of the original code into two separated domain hierarchies^[1], the HDL domain for the DUT and other accelerated HDL code, and the testbench domain for, well, the class-based testbench. Click [here](#) to see the original single top and corresponding dual top code in full. Most of the original code belongs in the HDL top level module. The exception is the UVM initial block to launch the UVM test, which remains in the testbench domain.

The HDL Domain

The DUT domain top-level module should contain:

- DUT instantiation
- DUT pin interface instantiations
- Clock and reset signal generation
- BFM interface instantiations

Interface Declarations and Instantiations

Transactor BFM interfaces and protocol signal interfaces are instantiated and interconnected in the traditional manner through direct signal port connections.

```
logic PCLK;
logic PRESETn;

apb_if APB (PCLK, PRESETn);
spi_if SPI (PCLK, PRESETn);

apb_driver_bfm APB_DRIVER(
    .PCLK      (APB.PCLK) ,
    .PRESETn   (APB.PRESETn) ,
    .PADDR     (APB.PADDR) ,
    .PRDATA    (APB.PRDATA) ,
    .PWDATA    (APB.PWDATA) ,
    .PSEL      (APB.PSEL) ,
    .PENABLE   (APB.PENABLE) ,
    .PWRITE    (APB.PWRITE) ,
    .PREADY    (APB.PREADY)
```

```

);
apb_monitor_bfm APB_MONITOR(
    ...
);
spi_driver_bfm SPI_DRIVER(
    .PCLK      (SPI.PCLK),
    .PRESETn   (SPI.PRESETn),
    .clk       (SPI.clk),
    .cs        (SPI.cs),
    .miso      (SPI.miso),
    .mosi      (SPI.mosi)
);
spi_monitor_bfm SPI_MONITOR(
    ...
);

```

For the attentive reader, while one could conceivably do the interconnection more elegantly using interface ports, like so

```

apb_if APB (...);

apb_driver_bfm APB_DRIVER(APB)
...

```

this would lead to LRM non-compliant usage when virtual BFM interfaces come into play. A pity really, and something to consider for the SystemVerilog standard body. That said, despite the availability of SystemVerilog interfaces, the use of signal ports as DUT module ports is so entrenched in the design community anyway and largely continues to prevail over interface ports.

Clock and Reset Generation

Veloce requires that clock and reset generation be modeled in separate initial blocks, each annotated with the same pragma comment (*//tbx clkgen*):

```

// tbx clkgen
initial begin
    PCLK = 0;
    forever begin
        #10ns PCLK = ~PCLK;
    end
end

// tbx clkgen
initial begin
    PRESETn = 0;
    #80 PRESETn = 1;
end

```

Virtual Interface Binding

With Veloce XRTL support for a so-called "vif binding block" as shown below (note the pragma comment), the HDL domain may effectively contain the (non-synthesizable) UVM-based code to put the various virtual BFM interface handles into the UVM configuration database to set up the virtual interface binding mechanism with the testbench domain.

```
// tbx vif_binding_block
initial begin
    import uvm_pkg::uvm_config_db;
    uvm_config_db #(virtual apb_driver_bfm)::set(null, "uvm_test_top",
$psprintf("%m.APB_DRIVER"), APB_DRIVER);
    ...
end
```

The Testbench Domain

The testbench domain contains the code to instantiate the test object and start the test environment phasing. It may also handle the virtual interface binding instead of doing this (more elegantly, but sometimes more restrictively) on the HDL-side as above (don't do it on both sides!).

```
initial begin
    uvm_config_db #(virtual apb_driver_bfm)::set(null, "uvm_test_top",
"top_hdl.APB_DRIVER" , top_hdl.APB_DRIVER);
    ...
    run_test();
end
```

Split Transactors

This example boasts two interfaces: an APB interface and a SPI interface. Each interface has a driver and a monitor transactor. These four transactors must each be split into a BFM-proxy pair ^[1], namely a synthesizable (acceleratable) BFM for the HDL domain and a TLM-level portion (representing the BFM) for the testbench domain. For each BFM task or function, all assignments to signals are converted into appropriate blocking or non-blocking assignments. In this example, the BFM tasks lend themselves to a coding style using implicit state machines.

Clock Synchronization

Note that all tasks in the APB and SPI BFMs must start by synchronizing to a clock edge, as required by Veloce XRTL.

Communication Format

The separated BFM and proxy of the split transactor can communicate through a BFM task call via a virtual BFM interface handle in the proxy. Veloce requires that the data format of the task arguments is in a packed form. When there is a lot of data to be transferred, a delegate class to perform the conversion between transaction object data and the BFM packed data is convenient:

APB:

```

class apb_seq_item_converter;

    static function void from_class(input apb_seq_item t, output
apb_seq_item_vector_t v);
        apb_seq_item_s s;
        s.addr  = t.addr;
        s.data  = t.data;
        s.we    = t.we;
        s.delay = t.delay;
        s.error = t.error;
        v = s;
    endfunction

    static function void to_class(output apb_seq_item t, input
apb_seq_item_vector_t v);
        apb_seq_item_s s;
        s = v;
        t = new();
        t.addr  = s.addr;
        t.data  = s.data;
        t.we    = s.we;
        t.delay = s.delay;
        t.error = s.error;
    endfunction

endclass: apb_seq_item_converter

```

SPI:

Data transfer over the SPI interface is rather primitive and hence the use of a conversion class seems overkill. The data is passed directly instead as individual BFM task arguments.

APB Agent

[Click here](#) to view the original APB agent code and the resulting split transactors for the driver and monitor.

This is relatively straightforward, putting the timed, signal-level code into tasks in the BFM.

APB Driver

All the signal-level code is moved to a task 'do_item()' of the BFM. The driver proxy calls this task and the transaction data is converted using a delegate class, as discussed above:

```

apb_seq_item_converter::from_class(req, req_s);
BFM.do_item(req_s, psel_index, rsp_s);
apb_seq_item_converter::to_class(rsp, rsp_s);

```

The initialization code in the original driver that runs once at the beginning of the run task is placed in an equivalent initial block in the BFM:

```

initial begin
    PSEL <= 0;
    PENABLE <= 0;
    PADDR <= 0;
end

```

APB Monitor

The monitor BFM uses a back-pointer to send the transaction data back to its testbench proxy. The testbench monitor proxy sets the value of the back-pointer in the `end_of_elaboration` phase:

```

function void apb_monitor::end_of_elaboration_phase(uvm_phase phase);
    BFM.proxy = this;
endfunction: end_of_elaboration_phase

```

The testbench monitor proxy just starts the BFM task named `run()` and defines the `write()` function called from the BFM:

```

task apb_monitor::run_phase(uvm_phase phase);
    BFM.run(apb_index);
endtask: run_phase

function void apb_monitor::write(apb_seq_item_s item_s);
    apb_seq_item item;

    apb_seq_item_converter::to_class(item, item_s);
    this.item.copy(item);
    ap.write(this.item);
endfunction: write

```

In the BFM, a back pointer is declared and named appropriately 'proxy'. When a transaction is observed from pin activity, inside the BFM's `run()` thread, it is sent back to the testbench by calling a function of the BFM's proxy through the back-pointer:

```

apb_monitor proxy; // pragma tbx oneway proxy.write

task run(int index); // pragma tbx xtf

    forever begin
        // Detect the protocol event on the TBAI virtual interface

        ...

        // Publish the item to the subscribers
        proxy.write(item);
    end
endtask: run

```

Note that the back-pointer declaration in the monitor BFM is annotated with a Veloce pragma comment:

```

// pragma tbx oneway proxy.write

```

While not required, this pragma comment can be seen as a Veloce compiler directive that is useful under certain circumstances to enable significant so-called one-way caller performance optimization. Effectively, the pragma comment directs the Veloce emulator to keep running, i.e. not halt the design clocks and not yield to testbench threads, at the time of the 'write()' function call, as would otherwise happen for the purpose of communication/synchronization with the simulator. This is feasible in particular because the 'write()' function is a strict one-way function - a void function without output arguments and without "side effects" (like a 'pure' void function in C/C++). The pragma can be used more generally in cases where an 'outbound' function call through the proxy back-pointer to the testbench only results in strict zero time passive (as opposed to reactive) consumption (indeed just like an analysis transaction from a monitor out via an analysis port to a scoreboard or coverage collector). Consult the Veloce user documentation for more information on this and other acceleration performance optimizations available.

SPI Agent

[Click here to view the original SPI agent code and the resulting split transactors for the driver and monitor.](#)

The original SPI driver and monitor pose a more significant challenge to split and maintain equivalent behavior. The challenge is not with the methodology per se, however, but specifically with the re-coding of the transactor behavior in an equivalent yet synthesizable manner.

The original SPI components use an interface-specific clock, 'SPI.clk'. The protocol code is sensitive to both edges of this clock. In order to make this Veloce synthesis compliant, all activity must be made synchronous to a single edge of a single clock. An auxiliary system clock is introduced for this purpose.

SPI Driver

The original driver starts out with code that does not adhere to the methodology rules (no timing control in the testbench side), **and** is also not synthesizable:

```
while(cs === 8'hxx) begin
    #1;
end
```

The HDL-side BFM interface defines an 'init()' task to implement this in a synthesis-friendly way:

```
task init(); // pragma tbx xtf
    @(negedge PCLK);
    while(cs != 8'hff) @(negedge PCLK);
endtask: init
```

A part of the original driver code that is sensitive to both edges of the SPI clock:

```
if(req.RX_NEG == 1) begin
    @(posedge clk);
end
else begin
    @(negedge clk);
end
```

This is rewritten for synthesis through clock synchronization with the negedge of PCLK, and using a task argument RX_NEG to handle and mimic sensitivity to both edges of the original SPI clk:

```
for(int i = 1; i < num_bits-1; i++) begin
    @(negedge PCLK); //--
```

```

    while(clk == RX_NEG) @(negedge PCLK); //  |- mimics if (RX_NEG == 1)
@ (posedge clk) else @(negedge clk)
    while(clk != RX_NEG) @(negedge PCLK); //--
    miso <= spi_data[i];
    if(cs == 8'hff) begin
        break;
    end
end
end

```

SPI Monitor

The SPI monitor is implemented in similar fashion as the APB monitor. A main 'run()' task is implemented in the BFM interface and when a transaction is observed, the BFM calls a 'write()' function in the testbench monitor proxy through a back-pointer. SPI clock synchronization handling is done in the same way as above for the SPI driver, by bringing in 'PCLK' as the system clock.

The main challenge of the monitor is handling the fork/join_any in the original code:

```

fork
begin
    while(cs != 8'hff) begin
        @(clk);
        if(clk == 1) begin
            item.nedge_mosi[p] = mosi;
            item.nedge_miso[p] = miso;
            p++;
        end
        else begin
            item.pedge_mosi[n] = mosi;
            item.pedge_miso[n] = miso;
            n++;
        end
    end
end
begin
    @(clk);
    @(cs);
end
join_any
disable fork;

```

There are two processes forked, one that collects bits from the serial stream and the other that watches the chip-select signal. Because of the join_any followed by disable fork, if the chip-select changes value at any time during the bit collection, the bit collection is aborted. Also, the sampling of the chip-select is done on both edges of the SPI clock.

This behavior is implemented in a synthesizable way in the BFM:

```

clk_val = clk; // --
@(negedge PCLK); //  |- mimics @(clk);
while(clk == clk_val) @(negedge PCLK); // --

```

```

while(cs == local_cs) begin
    if(clk == 1) begin
        nedge_mosi[p] <= mosi;
        nedge_miso[p] <= miso;
        p++;
    end
    else begin
        pedge_mosi[n] <= mosi;
        pedge_miso[n] <= miso;
        n++;
    end
    clk_val = clk;          // ---
    @(negedge PCLK);        //   |
    while(clk == clk_val) begin //   |- mimics @(clk) with premature
break on cs change
        @(negedge PCLK);    //   |
        if (cs != local_cs) break; // ---
    end
end
end

```

Remove Timing from Testbench Domain

The run_phase() task of the original SPI test class is as follows:

```

task spi_test::run_phase(uvm_phase phase);
    send_spi_char_seq spi_char_seq =
send_spi_char_seq::type_id::create("spi_char_seq");

    phase.raise_objection(this, "Starting spi_char_seq");
    spi_char_seq.start(m_env.m_v_sqr.apb);
    #100ns;
    phase.drop_objection(this, "Finished spi_char_seq");
endtask: run_phase

```

This code has a pound-delay timing control statement that violates the methodology rules. Such delays in the test or in sequences can be remodeled using a general-purpose delay task in an HDL-side interface - in this example, the task is in the INTR interface:

```

task wait_n_cycles(int n); // pragma tbx xtf
    @(posedge PCLK);
    assert(n>0);
    repeat (n-1) @(posedge PCLK);
endtask: wait_n_cycles

```

In order to call this task from any point in the testbench, one would need global access to the virtual interface handle. Using the technique described here, a "Wait_for_interface_signal" task can be defined, which can be called from places in the testbench other than the transactors:

```

// This task is a convenience method placed in the configuration object
// for sequences and tests waiting for time to elapse
task spi_env_config::pound_delay(int n);

```

```

if(n == 0) begin
    `uvm_error("SPI_ENV_CONFIG",
        $sformatf("Argument n for pound_delay must be greater than
zero"))
end
if (n % 20 == 0) begin
    INTR.wait_n_cycles(n);
end
else begin
    `uvm_warning("SPI_ENV_CONFIG",
        $sformatf("Argument n=%0d for pound_delay not a multiple of 20;
delay rounded up to next integer multiple %0d", n, (n/20+1)*20))
    INTR.wait_n_cycles(n/20+1);
end
endtask: pound_delay

```

Now, the test can be rewritten to use the configuration task instead of using the pound-delay statement:

```

task spi_test::run_phase(uvm_phase phase);
    send_spi_char_seq spi_char_seq =
send_spi_char_seq::type_id::create("spi_char_seq");

    phase.raise_objection(this, "Starting spi_char_seq");
    spi_char_seq.start(m_env.m_v_sqr.apb);
    m_env_cfg.pound_delay(100); // <----- Call to "wait_for_interface_signal" convenience method
    phase.drop_objection(this, "Finished spi_char_seq");
endtask: run_phase

```

Example Driver

Original 'Single Top' APB Driver

```

task apb_driver::run_phase(uvm_phase phase);
    apb_seq_item req;
    apb_seq_item rsp;
    int psel_index;

    forever begin
        APB.PSEL <= 0;
        APB.PENABLE <= 0;
        APB.PADDR <= 0;
        seq_item_port.get_next_item(req);
        repeat(req.delay)
            @(posedge APB.PCLK);
        psel_index = sel_lookup(req.addr);
        if(psel_index >= 0) begin
            APB.PSEL[psel_index] <= 1;
            APB.PADDR <= req.addr;
            APB.PWDATA <= req.data;
            APB.PWRITE <= req.we;
            @(posedge APB.PCLK);
            APB.PENABLE <= 1;
            while (!APB.PREADY)
                @(posedge APB.PCLK);
            if(APB.PWRITE == 0) begin
                req.data = APB.PRDATA;
            end
        end
        else begin
            `uvm_error("RUN", $sformatf("Access to addr %0h out of APB
address range", req.addr))
            req.error = 1;
        end
        seq_item_port.item_done();
    end

endtask: run_phase

```

Testbench Domain APB Driver Proxy

```

task apb_driver::run_phase(uvm_phase phase);
    apb_seq_item req;
    apb_seq_item rsp;
    int psel_index;

    forever begin

```

```

    apb_seq_item_s req_s, rsp_s;

    seq_item_port.get_next_item(req);
    psel_index = sel_lookup(req.addr);
    if(psel_index < 0) begin
        `uvm_error("RUN", $sformatf("Access to addr %0h out of APB
address range", req.addr))
    end
    apb_seq_item_converter::from_class(req, req_s);
    BFM.do_item(req_s, psel_index, rsp_s);
    apb_seq_item_converter::to_class(rsp, rsp_s);
    req.copy(rsp);
    seq_item_port.item_done();
end
endtask: run_phase

```

HDL Domain APB Driver BFM

```

interface apb_driver_bfm (input bit        PCLK,
                          input bit        PRESETn,
                          output logic [31:0] PADDR,
                          input logic [31:0] PRDATA,
                          output logic [31:0] PWDATA,
                          output logic [15:0] PSEL, // Only connect the
ones that are needed
                          output logic      PENABLE,
                          output logic      PWRITE,
                          input logic      PREADY);
// pragma attribute apb_driver_bfm partition_interface_xif

import apb_shared_pkg::apb_seq_item_s;

initial begin
    PSEL <= 0;
    PENABLE <= 0;
    PADDR <= 0;
end

task do_item(apb_seq_item_s req, int psel_index, output apb_seq_item_s
rsp); // pragma tbx xtf
    @(posedge PCLK);
    repeat(req.delay-1) // ok since delay is constrained to be between 1
and 20
        @(posedge PCLK);

    rsp = req;
    rsp.error = (psel_index < 0);
endtask

```

```

    if(rsp.error) return;

    PSEL[psel_index] <= 1;
    PADDR <= req.addr;
    PWDATA <= req.data;
    PWRITE <= req.we;
    @(posedge PCLK);
    PENABLE <= 1;
    while (!PREADY)
        @(posedge PCLK);
    if(PWRITE == 0)
    begin
        rsp.data = PRDATA;
    end
    PSEL <= 0;
    PENABLE <= 0;
    PADDR <= 0;
endtask: do_item

endinterface: apb_driver_bfm

```

Original 'Single Top' APB Monitor

```

task apb_monitor::run_phase(uvm_phase phase);
    apb_seq_item item;
    apb_seq_item cloned_item;

    item = apb_seq_item::type_id::create("item");

    forever begin
        // Detect the protocol event on the TBAI virtual interface
        @(posedge APB.PCLK);
        if(APB.PREADY && APB.PSEL[apb_index]) begin
            // Assign the relevant values to the analysis item fields
            item.addr = APB.PADDR;
            item.we = APB.PWRITE;
            if(APB.PWRITE) begin
                item.data = APB.PWDATA;
            end
            else begin
                item.data = APB.PRDATA;
            end
            // Clone and publish the cloned item to the subscribers
            $cast(cloned_item, item.clone());
            ap.write(cloned_item);
        end
    end
endtask: run_phase

```

Testbench Domain APB Monitor Proxy

```

function void apb_monitor::end_of_elaboration_phase(uvm_phase phase);
    BFM.proxy = this;
endfunction: end_of_elaboration_phase

task apb_monitor::run_phase(uvm_phase phase);
    BFM.run(apb_index);
endtask: run_phase

function void apb_monitor::write(apb_seq_item_s item_s);
    apb_seq_item item;

    apb_seq_item_converter::to_class(item, item_s);
    this.item.copy(item);
    ap.write(this.item);
endfunction: write

```

HDL Domain APB Monitor BFM

```

interface apb_monitor_bfm(input bit          PCLK,
                        input bit          PRESETn,
                        input logic [31:0] PADDR,
                        input logic [31:0] PRDATA,
                        input logic [31:0] PWDATA,
                        input logic [15:0] PSEL,
                        input logic          PENABLE,
                        input logic          PWRITE,
                        input logic          PREADY);

// pragma attribute apb_monitor_bfm partition_interface_xif

import apb_shared_pkg::apb_seq_item_s;
import apb_agent_pkg::apb_monitor;

apb_monitor proxy; // pragma tbx oneway proxy.write

task run(int index); // pragma tbx xtf
    apb_seq_item_s item;

    @(posedge PCLK);

    forever begin
        // Detect the protocol event on the TBAI virtual interface
        @(posedge PCLK);
        if(PREADY && PSEL[index]) // index identifies PSEL line this
monitor is connected to

        // Assign the relevant values to the analysis item fields
        begin

```

```

        item.addr = PADDR;
        item.we = PWRITE;
        if(PWRITE)
            begin
                item.data = PWDATA;
            end
        else
            begin
                item.data = PRDATA;
            end
        // Publish the item to the subscribers
        proxy.write(item);
    end
end
endtask: run

endinterface: apb_monitor_bfm

```

Example Agent

Original 'Single Top' SPI Driver

```

// This driver is really a SPI slave responder
task spi_driver::run_phase(uvm_phase phase);
    spi_seq_item req;
    spi_seq_item rsp;
    int no_bits;

    SPI.miso = 1;
    while(SPI.cs == 8'hxx) begin
        #1;
    end

    forever begin
        seq_item_port.get_next_item(req);
        while(SPI.cs == 8'hff) begin
            @(SPI.cs);
        end
        no_bits = req.no_bits;
        if(no_bits == 0) begin
            no_bits = 128;
        end
        SPI.miso = req.spi_data[0];
        for(int i = 1; i < no_bits-1; i++) begin
            if(req.RX_NEG == 1) begin
                @(posedge SPI.clk);
            end
        end
    end
endtask

```

```

        end
        else begin
            @(negedge SPI.clk);
        end
        SPI.miso = req.spi_data[i];
        if(SPI.cs == 8'hff) begin
            break;
        end
    end
    seq_item_port.item_done();
end
endtask: run_phase

```

Testbench Domain SPI Driver Proxy

```

// This driver is really a SPI slave responder
task spi_driver::run_phase(uvm_phase phase);
    spi_seq_item req;
    spi_seq_item rsp;

    BFM.init();

    forever begin
        seq_item_port.get_next_item(req);
        BFM.do_item(req.spi_data, req.no_bits, req.RX_NEG);
        seq_item_port.item_done();
    end
endtask: run_phase

```

HDL Domain SPI Driver BFM

```

interface spi_driver_bfm (input bit        PCLK,
                        input bit        PRESETn,
                        input logic      clk,
                        input logic [7:0] cs,
                        output logic      miso,
                        input logic      mosi);

// pragma attribute spi_driver_bfm partition_interface_xif

initial begin
    miso = 1;
end

task init(); // pragma tbx xtf
    @(negedge PCLK);
    while(cs != 8'hff) @(negedge PCLK);
endtask: init

// This driver is really an SPI slave responder

```

```

task do_item(logic[127:0] spi_data, bit[6:0] no_bits, bit RX_NEG); //
pragma tbx xtf
    bit[7:0] num_bits;

    @(negedge PCLK);

    while(cs == 8'hff) @(negedge PCLK);

    num_bits = no_bits;
    if(num_bits == 0) begin
        num_bits = 128;
    end
    miso <= spi_data[0];
    for(int i = 1; i < num_bits-1; i++) begin
        @(negedge PCLK); \
        while(clk == RX_NEG) @(negedge PCLK); > // mimics if (RX_NEG == 1)
    @(posedge clk) else @(negedge clk)
        while(clk != RX_NEG) @(negedge PCLK); /
        miso <= spi_data[i];
        if(cs == 8'hff) begin
            break;
        end
    end
endtask: do_item

endinterface: spi_driver_bfm

```

Original'Single Top' SPI Monitor

```

task spi_monitor::run_phase(uvm_phase);
    spi_seq_item item;
    spi_seq_item cloned_item;
    int n;
    int p;

    item = spi_seq_item::type_id::create("item");

    while(SPI.cs === 8'hxx) begin
        #1;
    end

    forever begin

        while(SPI.cs === 8'hff) begin
            @(SPI.cs);
        end

        n = 0;

```

```

    p = 0;
    item.nedge_mosi = 0;
    item.pedge_mosi = 0;
    item.nedge_miso = 0;
    item.pedge_miso = 0;
    item.cs = SPI.cs;

    fork
        begin
            while(SPI.cs != 8'hff) begin
                @(SPI.clk);
                if(SPI.clk == 1) begin
                    item.nedge_mosi[p] = SPI.mosi;
                    item.nedge_miso[p] = SPI.miso;
                    p++;
                end
                else begin
                    item.pedge_mosi[n] = SPI.mosi;
                    item.pedge_miso[n] = SPI.miso;
                    n++;
                end
            end
        end
    end
    begin
        @(SPI.clk);
        @(SPI.cs);
    end
    join_any
    disable fork;

    // Clone and publish the cloned item to the subscribers
    $cast(cloned_item, item.clone());
    ap.write(cloned_item);
end
endtask: run_phase

```

Testbench Domain SPI Monitor Proxy

```

function void spi_monitor::end_of_elaboration_phase(uvm_phase phase);
    BFM.proxy = this;
endfunction: end_of_elaboration_phase

task spi_monitor::run_phase(uvm_phase);
    item = spi_seq_item::type_id::create("item");
    BFM.run();
endtask: run_phase

function void spi_monitor::write(logic[127:0] nedge_mosi, pedge_mosi,

```

```

nedge_miso, pedge_miso, logic[7:0] cs);
    spi_seq_item cloned_item;

    item.nedge_mosi = nedge_mosi;
    item.pedge_mosi = pedge_mosi;
    item.nedge_miso = nedge_miso;
    item.pedge_miso = pedge_miso;
    item.cs = cs;
    // Clone and publish the cloned item to the subscribers
    $cast(cloned_item, item.clone());
    ap.write(cloned_item);
endfunction: write

```

HDL Domain SPI Monitor BFM

```

interface spi_monitor_bfm(input bit          PCLK,
                          input bit          PRESETn,
                          input logic        clk,
                          input logic [7:0] cs,
                          input logic        miso,
                          input logic        mosi);

// pragma attribute spi_monitor_bfm partition_interface_xif

import spi_agent_pkg::spi_monitor;

spi_monitor proxy; // pragma tbx oneway proxy.write

task run(); // pragma tbx xtf
    logic[127:0] nedge_mosi;
    logic[127:0] pedge_mosi;
    logic[127:0] nedge_miso;
    logic[127:0] pedge_miso;
    logic[7:0] local_cs;
    int n;
    int p;
    bit clk_val;

    @(negedge PCLK);

    while(cs != 8'hff) @(negedge PCLK);

    forever begin
        while(cs == 8'hff) @(negedge PCLK);

        n = 0;
        p = 0;
        nedge_mosi <= 0;
        pedge_mosi <= 0;

```

```

nedge_miso <= 0;
pedge_miso <= 0;
local_cs <= cs;

clk_val = clk;
@(negedge PCLK);
while(clk == clk_val) @(negedge PCLK); /

while(cs == local_cs) begin
    if(clk == 1) begin
        nedge_mosi[p] <= mosi;
        nedge_miso[p] <= miso;
        p++;
    end
    else begin
        pedge_mosi[n] <= mosi;
        pedge_miso[n] <= miso;
        n++;
    end
    clk_val = clk;
    @(negedge PCLK);
    while(clk == clk_val) begin > // mimics @(clk) with premature
break on cs change
        @(negedge PCLK); /
        if (cs != local_cs) break; /
    end
end

// Publish to the subscribers
proxy.write(nedge_mosi, pedge_mosi, nedge_miso, pedge_miso,
local_cs);
end
endtask: run

endinterface: spi_monitor_bfm

```

Example Top-Level Model

Original 'Single' Top-Level

```
module top_tb;

import uvm_pkg::*;
import spi_test_lib_pkg::*;

// PCLK and PRESETn
//
logic PCLK;
logic PRESETn;

//
// Instantiate the interfaces:
//
apb_if    APB(PCLK, PRESETn); // APB interface
spi_if    SPI();              // SPI Interface
intr_if INTR();              // Interrupt

// DUT
spi_top DUT(
    // DUT pin connections not shown...
);

// UVM initial block:
// Virtual interface wrapping & run_test()
initial begin
    uvm_config_db #(virtual apb_if)::set(null, "uvm_test_top", "APB_vif"
, APB);
    uvm_config_db #(virtual spi_if)::set(null, "uvm_test_top", "SPI_vif"
, SPI);
    uvm_config_db #(virtual intr_if)::set(null, "uvm_test_top", "INTR_vif", INTR);
    run_test();
end

//
// Clock and reset initial block:
//
initial begin
    PCLK = 0;
    PRESETn = 0;
    repeat(8) begin
        #10ns PCLK = ~PCLK;
```

```

end
PRESETn = 1;
forever begin
    #10ns PCLK = ~PCLK;
end
end
endmodule: top_tb

```

HDL Domain Top-Level

```

module top_hdl;

`include "timescale.v"

// PCLK and PRESETn
//
logic PCLK;
logic PRESETn;

//
// Instantiate the interfaces:
//
apb_if APB(PCLK, PRESETn); // APB interface
spi_if SPI(PCLK, PRESETn); // SPI Interface

intr_if INTR(PCLK, PRESETn); // Interrupt

// tbx vif_binding_block
initial begin
    import uvm_pkg::uvm_config_db;
    uvm_config_db #(virtual intr_if)::set(null, "uvm_test_top",
    $psprintf("%m.INTR") , INTR);
end

apb_driver_bfm APB_DRIVER(.PCLK(APB.PCLK),
                        .PRESETn(APB.PRESETn),
                        .PADDR(APB.PADDR),
                        .PRDATA(APB.PRDATA),
                        .PWDATA(APB.PWDATA),
                        .PSEL(APB.PSEL),
                        .PENABLE(APB.PENABLE),
                        .PWRITE(APB.PWRITE),
                        .PREADY(APB.PREADY));
apb_monitor_bfm APB_MONITOR(.PCLK(APB.PCLK),
                        .PRESETn(APB.PRESETn),
                        .PADDR(APB.PADDR),
                        .PRDATA(APB.PRDATA),

```

```

        .PWDATA(APB.PWDATA),
        .PSEL(APB.PSEL),
        .PENABLE(APB.PENABLE),
        .PWRITE(APB.PWRITE),
        .PREADY(APB.PREADY));

// tbx vif_binding_block
initial begin
    import uvm_pkg::uvm_config_db;
    uvm_config_db #(virtual apb_driver_bfm)::set(null, "uvm_test_top",
$psprintf("%m.APB_DRIVER") , APB_DRIVER);
    uvm_config_db #(virtual apb_monitor_bfm)::set(null, "uvm_test_top",
$psprintf("%m.APB_MONITOR") , APB_MONITOR);
end

spi_driver_bfm      SPI_DRIVER(.PCLK(SPI.PCLK),
        .PRESETn(SPI.PRESETn),
        .clk(SPI.clk),
        .cs(SPI.cs),
        .miso(SPI.miso),
        .mosi(SPI.mosi));
spi_monitor_bfm SPI_MONITOR(.PCLK(SPI.PCLK),
        .PRESETn(SPI.PRESETn),
        .clk(SPI.clk),
        .cs(SPI.cs),
        .miso(SPI.miso),
        .mosi(SPI.mosi));

// tbx vif_binding_block
initial begin
    import uvm_pkg::uvm_config_db;
    uvm_config_db #(virtual spi_driver_bfm)::set(null, "uvm_test_top",
$psprintf("%m.SPI_DRIVER") , SPI_DRIVER);
    uvm_config_db #(virtual spi_monitor_bfm)::set(null, "uvm_test_top",
$psprintf("%m.SPI_MONITOR") , SPI_MONITOR);
end

// DUT
spi_top DUT(
    // APB Interface:
    .PCLK(PCLK),
    .PRESETN(PRESETn),
    .PSEL(APB.PSEL[0]),
    .PADDR(APB.PADDR[4:0]),
    .PWDATA(APB.PWDATA),
    .PRDATA(APB.PRDATA),
    .PENABLE(APB.PENABLE),

```

```

    .PREADY (APB.PREADY),
    .PSLVERR(),
    .PWRITE (APB.PWRITE),
    // Interrupt output
    .IRQ (INTR.IRQ),
    // SPI signals
    .ss_pad_o (SPI.cs),
    .sclk_pad_o (SPI.clk),
    .mosi_pad_o (SPI.mosi),
    .miso_pad_i (SPI.miso)
);

//
// Clock and reset initial blocks:
//
// tbx clkgen
initial begin
    PCLK = 0;
    forever begin
        #10ns PCLK = ~PCLK;
    end
end

// tbx clkgen
initial begin
    PRESETn = 0;
    #80 PRESETn = 1;
end

endmodule: top_hdl

```

Testbench Domain Top-level

```

module top_tb;

import uvm_pkg::*;
import spi_test_lib_pkg::*;

// UVM initial block: run_test()
initial begin
    run_test();
end

endmodule: top_tb

```